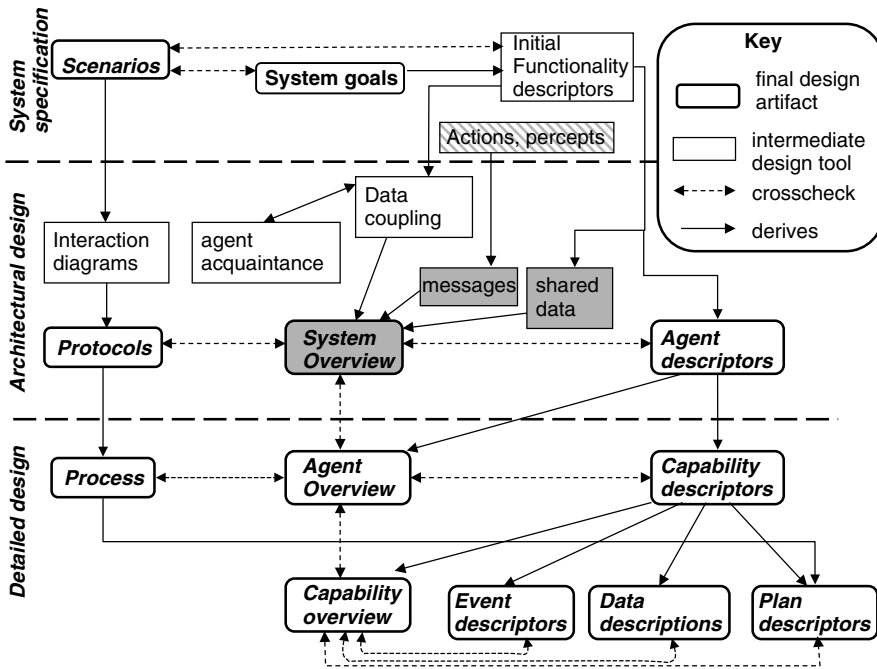


Finalizing the Architectural Design



Phases, artifacts and relationships in the design process

Having specified the agents within the system and the communications between them, we now bring this information together in a *System Overview Diagram*, which captures diagrammatically the overall architecture of the agent system. In order to do this, we also need to develop a fuller understanding of the data needed as well as of the environmental

information that will be responded to and the actions that the agent may take within the environment.

7.1 OVERALL SYSTEM STRUCTURE

In building a multi-agent system, it is often the case that some portions of the overall system are more appropriately developed outside of an agent paradigm. Such aspects typically fall into one of four categories:

- Data representation and management
- Percept processing
- Action management
- Pre-existing code.

In this final step of the System architecture phase (see Figure 5.1, Page 54), we

1. identify the boundaries of the agent system and the interactions with other sub-systems;
2. describe the percepts and actions, and the relationships between these and relevant agents;
3. define all shared data, both external persistent data and internal shared data within the system;
4. develop the system overview diagram.

7.2 IDENTIFYING BOUNDARIES OF THE AGENT SYSTEM

Agent systems are always operating within some environment, and thus the specification of the interfaces to that environment are very important. The boundaries of the agent system may be the physical environment, but there are also quite likely to be software boundaries where the agent system forms part of a larger application.

One part of the interface is the information coming from the (usually dynamic) environment, which we have called *percepts*. Depending on the application, the raw data from the environment may require significant processing in order to be of any use to the agents in the system. It may well be more appropriate for this processing to take place in a non-agent sub-system, than to incorporate it within an agent. Reasons for this may include existence of more suitable languages than that being used for agent development, lack of benefit of the agent paradigm or pre-existing libraries that can be used.

For example, in an agent system controlling a robot, the perceptual input may be vision data from a camera, in the form of a series of bitmaps. However, this requires significant pre-processing before it is of interest to the agent. Firstly, there must be

some processing to identify relevant objects within the bitmap. However, even this is not really sufficient. In order to act within the environment, the agent wants to notice when interesting or significant things have happened. It does not want to reason about every frame that is essentially the same as the previous one. Typically, it will want to notice differences between frames, particularly unexpected differences. It may also notice differences between frames and its expectations – for example, if an agent believes, on the basis of previous information, that a ball is just to its left, but when it turns and looks at that spot it does not see a ball, then this is significant information.

We use the term *percept* to refer to the environmental information that the agent receives, although we acknowledge that this may well have received significant processing from the raw data obtained.

Similarly, there may be some aspects of managing the agent's actions in the environment, which are best performed outside of an agent paradigm. Using our robot example again, a typical action may well be to move to a specified point. However, there is a significant amount of motor control programming required in order to effectively implement this. This is most appropriately seen as a behavioural sub-system for each agent, which is designed and implemented outside of the agent paradigm being described here.

It may also be the case that pre-existing code performing some relevant functionality is to be incorporated into the agent system. The interface between such sub-systems and the agent system must also be identified. This can be represented in terms of messages, which may be requests and responses, but may also include incoming messages generated by the external sub-system if it runs asynchronously and can generate data other than as a synchronized response to a request. Alternatively, the external sub-system can be seen as part of the environment, and interaction can be represented in terms of percepts and actions.

The implementation of the interface to an external sub-system may involve wrapping pre-existing code as agents and using agent messaging, or may simply be the use of a defined API. This decision is not important at design time and is left as an implementation decision. However, the information needed and provided by the pre-existing sub-system is important and should be documented as part of either messages or percepts and actions.

Issues to Consider

Some issues to consider when making the decision regarding whether to include something within the agent system or to make it a separate sub-system are as follows:

1. Is there any benefit (currently or in future extensions of the system) of the agent paradigm with aspects such as autonomy, flexibility and goal orientation?
2. Does keeping it separate make for less coupling, allowing it to be developed separately?
3. What is the simplest approach to integrating, or interfacing to, existing code?

4. Does the processing in question need to be on a different time cycle than the agent processing? For example, processing frames from a video camera may need to be done 50 times a second, whereas higher-level decision-making may take place at a much lower rate (such as five times a second).

7.3 DESCRIBING PERCEPTS AND ACTIONS

In this step, we develop descriptors for the input and output interface entities, percepts and actions, and link these to relevant agents.

Percepts are the information (possibly processed) that the agent receives from the environment, while actions represent the effects the agent can have on the external environment. We discuss these, identifying issues to consider and providing a template for recording of important information for understanding the design.

 **TIP:** Although the exact details of the interface to the external environment may be considered as implementation, or perhaps detailed design, it is critical to establish some aspects of this at an early stage. If it is impossible, or very difficult, to obtain particular data, then this will affect the design of the system at a high level.

PERCEPTS

It is important at this stage to clarify exactly what information is provided as dynamic input to the agent system. Most percepts will be a result of ongoing events happening in the external environment, but some may also have to do with initialization of the agent system. For example, a percept ‘TCP/IP connection established’ may be something that will be noticed by the agent at system start-up and which should cause some agents to read in data and establish internal data structures.

Identification of percepts to be obtained is driven by the information the agent system requires in order to achieve the goals identified in the initial phase. However, this is also constrained by what is feasible given the sensors or other mechanisms for obtaining data and the processing techniques available. Where substantial processing of raw data is required, great care should be taken to ensure that extraction of relevant information is feasible, prior to continuing the design of the agent system.

It is important to identify the format of the raw data, including whether there are any important intermediate formats of relevance for the pre-processing and extraction of agent percepts. For example, the vision frame from the robot camera has an initial format as a bitmap of a certain size. An intermediate format is a data structure containing certain objects. If existing software is being used, this data structure may have pre-defined contents or limitations, which are important to understand. It is then important to define the information contained in the final processed percept, as well as any additional information that may be required to obtain the desired percept.

For example, suppose we want to extract a percept ‘ball at position X, Y’ from vision data, with an intermediate representation that provides objects and their global location

information. We want this percept generated only when it is new information, not at every frame. In order for this to happen, we must at least maintain information regarding (one or more) previous frames. In addition, we may need access to information regarding what the agent currently believes – if it has momentarily turned so that the ball is out of sight, then turns back, it may be inappropriate to generate a percept when the ball is perceived in its expected location, despite the fact that it was not seen in the previous frame. On the other hand, failure to see the ball as expected may require percept generation.

It is also important to consider the likely frequency of such percepts and whether this is manageable by the agent system. If, for example, it is possible that a certain type of percept may on occasion happen with too high a frequency to be managed, then this may be a reason for some pre-processing. The percept could in normal cases be passed on directly, but in cases of high frequency we might need to either pass on only some percentage of the percepts, or aggregate percepts before passing them on.

Many percepts will result in an update of the agent's knowledge as well as potential action on the part of the agent. Knowledge updates resulting from the percept should be explicitly identified.

Our template descriptor for percepts is as follows:

- **Name** – Short name suitable for use in diagrams.
- **Description** – Brief description indicating the situation in which this percept is received.
- **Information carried** – Indicates what information is available regarding this percept. This often requires careful checking regarding the actual interface to the external environment.
- **Knowledge updated** – This will be either directly extracted from the information carried, or will be a result of some reasoning about the information carried.
- **Source** – Indicates how the information is obtained from the environment. In physical systems, this is likely to be a sensor.
- **Processing** – Indicates whether the percept is received directly from the agent sensor, or whether it is first processed in software, and if so how.
- **Agents responding** – Indicates which agents react to this percept in some way.
- **Expected frequency** – Describes how often one would expect to receive percepts of this type, with additional information regarding handling of extreme cases if appropriate.

Electronic Bookstore: Case study

An example of a percept from the electronic bookstore example is *Failed Stock Arrival*.

- **Name** – Failed Stock Arrival.

- **Description** – Occurs when stock is expected to arrive at a certain time, but the time passes (with some margin) and it does not arrive.
- **Information carried** – The reference to the stock order.
- **Knowledge updated** – Reliability of supplier, expected arrival date for waiting orders, expected arrival date for stock.
- **Source** – An indication from a previously set system alarm clock. (The stock orders DB is also accessed for information regarding the particular order.)
- **Processing** – When the ‘alarm’ goes off, records may need to be checked to ensure that the order has not in fact arrived, or been cancelled, or been previously notified.
- **Agents responding** – *Stock Manager* reacts to percept and notifies *Delivery Manager* if necessary, which may in turn notify *Customer Relations*.
- **Expected frequency** – Occasional.

 **TIP:** Percept processing is often an area where some prototyping is necessary at an early stage to avoid development of a design that is infeasible because of the difficulty of reliably extracting the required information from the raw data.

ACTIONS

Actions, like percepts, may also be complex, requiring significant design and development outside the realm of the reasoning system. This is especially true when manipulation of physical effectors is involved.

Actions can be classified depending on a number of properties:

- Is the action effectively instantaneous, or does it take a certain amount of time to perform (durational)?
For example, sending an e-mail is an effectively instantaneous action. Following a ball is a durational action. Another durational action might be downloading a large file.
- If the action is durational, does the action have parameters that can be changed while it is executing?
Downloading a file usually does not allow any parameters to be set – one simply waits for the download to complete. On the other hand, following a ball has a number of parameters including speed, the amount of acceleration that should be applied (for example, it might be desirable to conserve energy by using lower

acceleration levels), and so on. These factors apply to the design of the actions (outside of the agent part of the system) but also affect the design of agents, functionalities and plans that use these actions.

- Can the action fail? Most actions involving the real world can fail. Even in a software-only system, failures can occur: an e-mail address may be wrong, a web-site may be down, a network connection may be temporarily out of action.
- If the action can fail, does it always report failure when it occurs?
- Can the action have partial effects? For example, an e-mail is either delivered or not delivered. An attempt to move to a location may fail if the robot gets stuck halfway, but will still have changed the state of the world.

If actions are durational, that is, they take time, it is important to consider whether it is appropriate for the agent to do other tasks while this action is executing. In robot soccer, moving to a location is durational. While it is not appropriate for the agent to do other actions whilst doing this as it uses all the effectors, and any other action would interfere, it is important that the agent is free to do reasoning tasks on the basis of new perceptions.

In our electronic bookstore example, most of the actions are effectively instantaneous, such as sending an e-mail. Some, such as querying the status of a delivery, can be viewed either as a durational action that includes waiting for a reply, or as an instantaneous action of sending an e-mail (where receipt of the reply lies outside the action).

Our action descriptor template is as follows:

- Name
- Description – Indicates what the action is and its intended result.
- Parameters – Identifies parameters that can affect how the action is performed. For example, which printer to use may be a parameter to an action to print a file. Which speed to use may be a parameter for a robot move-to action.
- Temporality – Durational/instantaneous. If durational, indicate approximate length of time.
- Failure detection – Is there a notification if failure occurs (yes/no/maybe)? If so, what form does that notification take? If there is no notification, is it possible to check for success/failure in some way?
- Partial change – If the action does not succeed, what, if any, is the change that results from doing (or attempting) the action?
- Side effects – Indicates the side effects of an action, for example, the side effect of a move-to action may be a reduction in battery power.

Electronic Bookstore: Case study

An example action from the electronic bookstore is to order new stock:

- **Name** – Order stock
- **Description** – This action orders a list of books, in varying quantities, from a single supplier. The result of the action is that the supplier receives the order. An expected consequence is that the supplier sends a message confirming the order, and that eventually the books arrive. The latter are indirect results of the action rather than direct results.
- **Parameters** – Supplier e-mail address(es), order list, priority.
- **Temporality** – Instantaneous. (Indirect effects may take some days.)
- **Failure detection** – E-mail bounce may be detected. Only way to check success is to send e-mail requesting confirmation. Failure may go undetected.
- **Partial change** – Mail message file may exist.
- **Side effects** – None.

The percepts and actions provide the necessary interface between the system and its larger environment. To complete our understanding of the system at this level, we also need to include data sources.

7.4 DEFINING SHARED DATA OBJECTS

During the system specification stage, the persistent data has been identified. Now decisions must be made about what kind of persistent data stores will be used – a database or a file, and earlier ideas about which data items will be grouped together in a single data store must be confirmed and developed.

It must also be determined whether there will be any shared data within the system, accessible to more than one agent. A good design will minimize this, but there may be situations where it is reasonable to have shared data objects.

If multiple agents will be writing to shared data objects, this will require significant additional care for synchronization (as agents operate concurrently with each other). Consideration must be given as to whether this can potentially result in inconsistent data states containing parts of two different updates. This will depend partly on details of the implementation environment. Simple forms of locking can be used. We do not discuss locking further here since it is a standard topic in concurrent programming. However, it is important to ensure that locks are released both on success and failure.

☞ **TIP:** A common problem in multi-agent systems, which can be especially hard to debug, is *race conditions*. Data stores that are used by more than one agent can easily lead to race conditions because of synchronization issues. This can result in nondeterministic execution where the results of running the system can differ, even if the inputs are identical. This is one of many good reasons to avoid shared data stores.

A race condition is where the behaviour of a program depends on timing. Usually, such behaviour is unexpected. For example, consider two people, Alice and Bill, trying to update a bank account by reading the amount of money, adding 100 to it, and saving the result back to the account. The desired (and expected) result is that the account balance will be increased by 200. However, if the following sequence takes place, then the account will be incremented by only 100:

1. Alice reads the account balance (say, 150)
2. Bill reads the account balance (150)
3. Bill computes $\text{balance} + 100$ (i.e. $150 + 100 = 250$)
4. Bill saves $\text{balance} + 100$ (i.e. 250) to the account
5. Alice computes $\text{balance} + 100$, that is, $150 + 100$ (note that the original balance is being used, since that is what Alice read in step 1)
6. Alice saves $\text{balance} + 100$ (i.e. 250) to the account, overwriting Bill's change.

Database systems use transactions to avoid these problems, but such mechanisms are not always available or appropriate in agent systems.

Most applications have significant amounts of data with complex structure. If this data is to be stored within an external database, then it is appropriate to use database design techniques for specifying the design. If the data represents instances of interrelated classes that will be kept in main memory and possibly stored externally in some file format, then it is appropriate to use object-oriented design. The design should be developed and documented during this and subsequent phases, but we do not provide details here as there are already plenty of available materials providing methodology. Data may also have different internal and external representations.

Some important questions to ask at this stage about persistent data objects are as follows:

- Is there an in-memory version of the data?

- When is the external data updated – whenever an in-memory version changes, or not?
- Are any consistency checks needed on this data – if so what?
- How do agents access this data? (via requests to another agent, via DB access, reading a file, direct access to attributes of data object, via methods on a data object, and so on.)

It is important to note that when we talk about data, we need to differentiate data objects from data types. For example a hospital system may have a patient database which contains a set of patient records. There may also be a tuberculosis register which all hospitals in the state are required to use, also containing a set of patient records. It is not sufficient in the design to show these both as being of the same type, *patient records database*. To do so would have the effect of showing only one copy in design diagrams, which is very confusing, as they serve very different purposes. Rather, the two databases should be separated at design time, but would later be implemented as instances of the same class. Saying that we have a *tuberculosis database* and a *hospital patients database* provides useful information. Thus data stores that are collections of records are usually captured at the instance level on design diagrams, rather than at the type level as for other entities.

For shared data that is not persistent – that is, it exists only at run time, the main issues (other than the structure of the data), have to do with potential inconsistencies. If multiple agents can write data, what mechanisms prevent potential destruction of data? Are there any intermediate points when data is being modified, where it should not be accessed, due to incompleteness or inconsistency?

It is important to record with the documentation of the data, relevant information that is also available elsewhere in the design. This includes which agents read and write the data.

Our template for data descriptors is as follows:

- Name
- Description – What this data object represents.
- Data type – This should refer to part of a class diagram, or a database view.
- Included fields/aspects – What data, (perhaps named as necessary for certain functionalities) is considered part of this data object (or corresponding class).
- Produced by – Agent(s), capabilities, plans.
- Used by – Agent(s), capabilities, plans.
- Persistent – Indicates whether it is stored on an external device, file, database, etc.
- Initialization – What is initialization process or values.
- Used when – Indicates what the data is used for and when it is updated.

 **TIP:** Often, what at first appears to be a shared data object can be re-conceptualized as a data source managed by a single agent, with information provided to other agents as they need it. Alternatively, each agent may have its own version of the information, without there being any need for a single centralized data object.

7.5 SYSTEM OVERVIEW DIAGRAM

The above steps have identified all the pieces of the system architecture. There are two artifacts that collect this information together – the system overview diagram and the data dictionary.

The system overview diagram is arguably the single most important artifact of the entire design process, although of course it cannot really be understood fully in isolation. The various descriptors provide the more detailed information that may be required.

The symbols used in the system overview diagram are shown in Figure 7.1.

We start by placing icons for each percept type and action type as well as external data stores, at the boundaries of the figure. If there is a sub-system that is processing data to provide (some of) the percepts prior to them being provided to the agents, then this sub-system can be shown as a box (or multiple boxes), with the percepts originating from within it. Similarly, a non-agent sub-system that manages the details of agent actions can be shown as a box between the agent system entities producing the actions, and the actual actions.

We place within the figure a named agent symbol for each agent type. We then link percepts to agents that use them, and actions to agents that are responsible for them. We also place an incoming link from each agent that writes to an external data store and an outgoing link from each external data store to each agent that directly accesses its data. Double-headed links (arrows at both ends) indicate both read and write.

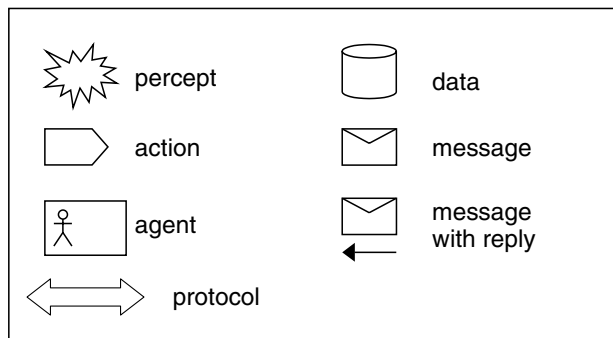


Figure 7.1 Graphical symbols used in the system overview diagram

Once we have arranged the agents and the interface entities, we add shared internal data stores, interaction information between agents and information regarding any additional external sub-systems.

We next place all protocols into the figure, by placing a named protocol link between each pair of agents that communicate via at least one message within a given protocol. If there are additional agent communications that exist outside the specified protocols, we indicate these by placing a named message symbol and linking to and from the relevant agent. If the communication is a message and response pair, then this is shown using the special ‘message with reply’ symbol.

If external code is being incorporated into the system, this is shown by a box with appropriate message or percept/action interface to some agent(s).

The example system overview diagram in Figure 7.2 depicts the four agents Agent1, Agent2, Agent3 and Agent4. Agent1 initiates ProtocolA involving also Agent3 and Agent2, as well as participating in ProtocolB. It also receives a message from Agent2 and percept P1 from the environment. It reads and writes database DB1. Agent2, as well as participating in ProtocolA and sending a message to Agent1, also interacts with an

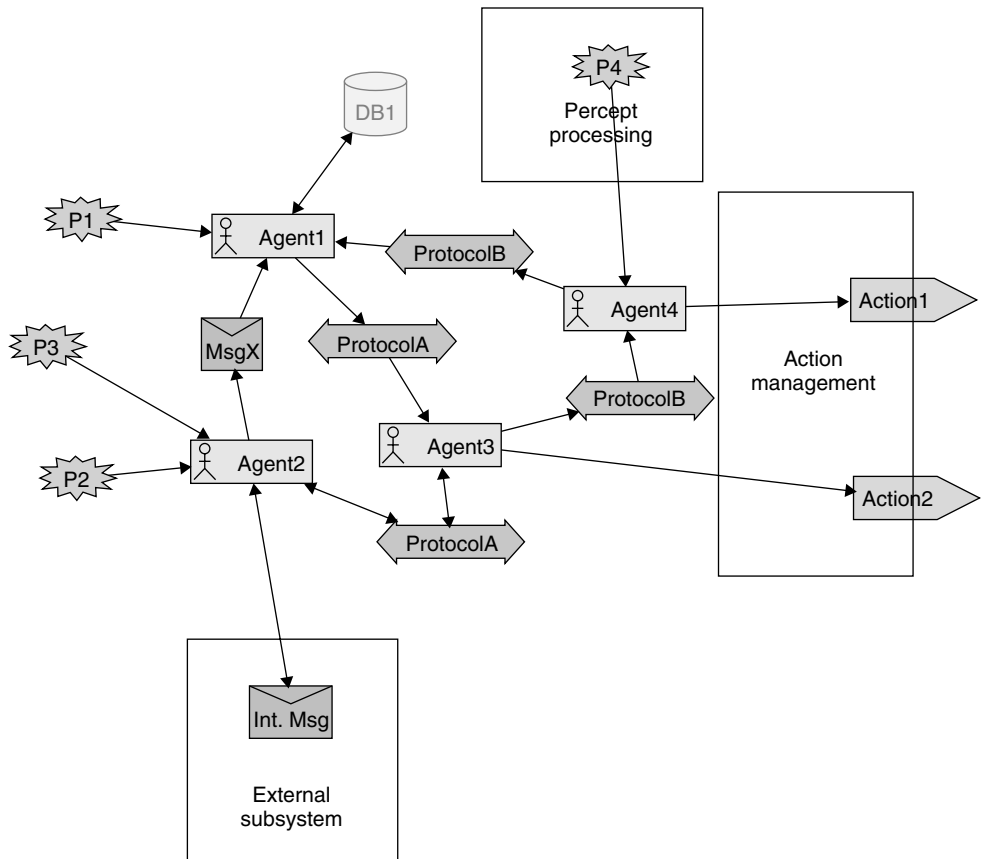


Figure 7.2 Example of system overview diagram

external sub-system via an interface message, and receives two percepts, P2 and P3. Agent3 executes Action2 as well as initiating ProtocolB and participating in ProtocolA. Agent4 receives percept P4 and does Action1, as well as participating in ProtocolB. As only external and shared databases are shown at this level, we see that DB1 is an external database, (read and written by Agent1).

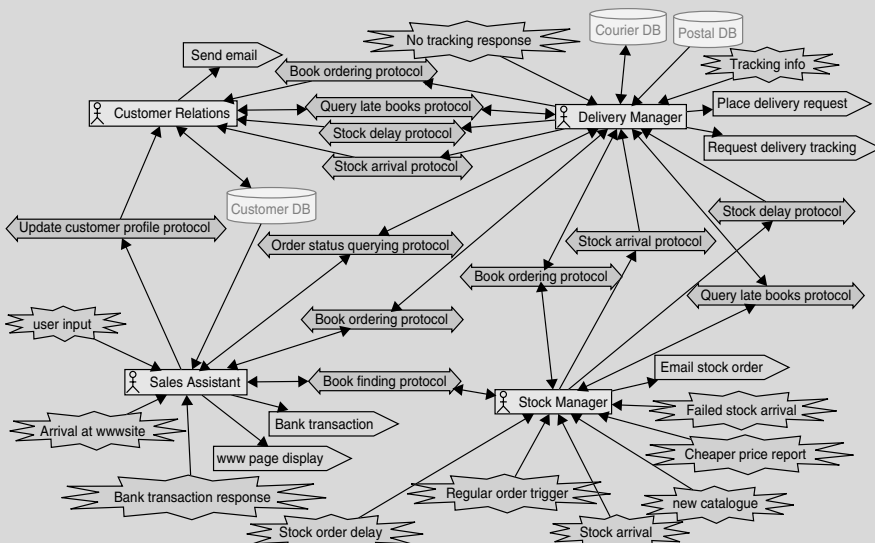
Electronic Bookstore: Case study

The following is a list of percepts, actions and protocols for the bookstore example, followed by the system overview diagram for the example.

Actions: Bank transaction, E-mail stock order, Place delivery request, Request delivery tracking, Send e-mail, WWW page display.

Percepts: Arrival at WWWsite, User input, Bank transaction response, Cheaper price report, Stock arrival, Failed stock arrival, Stock order delay, Regular order trigger, Tracking info, No tracking response, New catalogue.

Protocols: Book finding, Book ordering, Order status querying, Query late books, Stock arrival, Stock delay, Update customer profile.



System overview diagram for electronic bookstore

Note that the system overview diagram shows agent *types*, not agent instances.

If desired – and depending on the application – it is possible to provide additional overview diagrams showing agent instances, and partial views of system functionality. Often, these partial views would be focussing on a particular scenario or protocol.

Data Dictionary

The data dictionary is a list of all entities in the design. A data dictionary should be started at the beginning of the project and developed further at each stage.

A data dictionary should be maintained to allow for easy organization and location of the descriptors for the various entities.

One option is to organize the data dictionary into separate sections for agents, capabilities, plans, events and data, organized alphabetically within sections. The other option is to have a flat alphabetical structure. With tool support multiple views (automatically generated) can be provided as well as indexing.

 **TIP:** It is useful to develop the data dictionary in such a way that it can be sorted by name, by type, or by location defined. A spreadsheet can be a useful tool here.

Electronic Bookstore: Case study			
An excerpt of the data dictionary for the book store is below.			
Name	Type	Page	Notes
Bank transaction	action	page 160	
...			
Online interaction	functionality	page 141	one per customer
Customer DB	database	page 187	one in system
...			
Books DB	database	page 187	one in system
...			

7.6 CHECKING FOR COMPLETENESS AND CONSISTENCY

Even more than with the system specification phase, there are a number of ways in which the design should be checked for consistency and completeness. This consistency checking is also likely to incorporate some revision of artifacts from the system specification phase. As with system specification, naming consistency is the most basic check.

The additional checking at this stage falls into a number of main categories:

- consistency between agent descriptors and the descriptors of included functionalities;
- consistency between interaction diagrams, scenarios and protocols;
- consistency of communication specifications: that is, between protocols, system overview and agent descriptors and
- consistency between the descriptors and the system overview diagram.

The first two of the above are essentially vertical checks – ensuring that consistency is maintained with work at an earlier phase. The second two are horizontal checks ensuring consistency between different views within the same phase.

7.6.1 *CONSISTENCY BETWEEN AGENTS AND FUNCTIONALITIES*

The first check is to ensure that each functionality is assigned to exactly one agent.¹

Consistency between functionality descriptors and agent descriptors requires a number of basic checks. For the goals and actions fields, the agent descriptor field should be the union of the values in the equivalent field in the functionality descriptors of the included functionalities. Information used and produced by functionalities should be checked against data read, data written and internal data of the agent. This may not be a check that can be automated as apparent inconsistency can result from the fact that during architectural design, early thoughts about data needs have been revised and data representations have developed greater clarity and specificity. However, it is important to check that data needs identified during functionality specification are not inadvertently dropped. Depending on resources and the level of tool support, it may be desirable to update functionality descriptors in the light of further design work. However, as they are not regarded as a final design artifact, necessary for understanding of the final system, this may not be worthwhile.

7.6.2 *CONSISTENCY BETWEEN INTERACTION DIAGRAMS, SCENARIOS AND PROTOCOLS*

Scenarios, interaction diagrams and protocols are closely related, as is evident from the process described earlier for generating both interaction diagrams and protocols. Things that should be checked are as follows:

- Each scenario should have a corresponding interaction diagram, unless all functionalities involved in the scenario belong to the same agent. It is helpful if naming of interaction diagrams indicates clearly which scenario they are derived from, or if they are tagged in some way to provide this information.
- The interaction diagram corresponding to a given scenario should obey the following rules:
 - For each functionality mentioned in the scenario, the corresponding agent should exist in the interaction diagram.
 - Each agent in the interaction diagram should receive some information (either from another agent or via a percept or trigger) prior to sending out any message. The interaction diagram should really be manually checked to ensure

¹Possibly it is reasonable to have functionalities that are included in multiple agents. However, at the system specification phase, the authors have never actually found this useful. If required, it is a fairly straightforward modification. Of course at detailed design there will very likely be pieces of code – we call them *capabilities* – which will be included or reused in multiple agents.

that an agent has received the relevant information before each interaction – that is, that the information flow makes sense. (This cannot be automated without requiring more detail than is appropriate at this stage of the design.)

- Every interaction diagram that is more than a single message (possibly with response) should be represented within some protocol. This can be checked by mapping each interaction diagram to a scenario and then checking that it is in the *Included scenarios* field of some protocol descriptor. (Single interactions may be shown either by a degenerate protocol specification or by an individual message or message + reply).
- A protocol that is not related to any scenarios should probably result in returning to the system specification phase to specify some scenarios for design completeness. Scenarios are an important part of the final design document for understandability.
- The protocol describing a certain interaction should cover all possibilities. Specifically, if a protocol includes a use case scenario, then it should cover all of the scenario's variations. By 'cover' we mean that the sequence of messages in the interaction diagram corresponding to the use case scenario is one of the sequences that are allowed by the protocol.

7.6.3 CONSISTENCY OF COMMUNICATION SPECIFICATIONS

Communication between agents is specified in three different places within the architectural design. These are within protocols, within the system overview diagram and within agent descriptors. These three specifications need to be consistent – and in some cases, consistency can be actively maintained by a tool during development.

For example, the prototype Prometheus Design Tool ensures that if a protocol defines message passing between two agents, then there is a corresponding link indicated in the system overview diagram. Sometimes, however, it is practical only to automate consistency maintenance in one direction. For instance, it is easy to ensure that a protocol step showing a message between two agents results in a protocol linking these two on the system overview. However, it is less clear that it is desirable to automatically insert a message into a protocol on the basis of the designer indicating a protocol link between two agents. There is insufficient information to completely automate this step – we do not know what message is sent or when in the protocol it should happen – so it may be preferable simply to use the information to direct consistency checking at the end of the phase.

The particular consistencies we want to ensure are as follows:

- If there is a protocol link between two agents on the system overview, then there should be some message between these two agents in the protocol, and the protocol should be mentioned in both agent descriptors. Similarly, if there is a message between two agents in a protocol, it should be reflected in descriptors and the system overview diagram.
- If an agent descriptor lists that the agent *interacts with* another agent type, then that interaction should be evident in some protocol descriptor (or some single message

or pair of messages if protocols are not used for this case). Similarly, all agents that are interacted with according to protocol specifications should be listed in the agent descriptor.

7.6.4 *CONSISTENCY BETWEEN DESCRIPTORS AND THE SYSTEM OVERVIEW DIAGRAM*

There is considerable overlap in the information that is conveyed by the agent descriptors and that conveyed by the system overview diagram. The descriptors contain significant additional information, but for facilitating completeness, they also contain information that can be extracted from other design artifacts. Again, if an automated tool is used, much of this is trivial. If not, it is still straightforward, but surprisingly necessary in order to ensure that the design is indeed consistent. We have already covered above the agent interaction issues that need to be checked between descriptors and system overview. Additional items are as follows:

- Ensure that each agent is represented in the system overview diagram.
- Ensure that all persistent data stores are shown in the system overview and that reads and writes are consistent with agent descriptors, as well as data descriptors.
- Ensure that all percepts are shown in the diagram and are appropriately linked to agents, consistent with agent descriptor declarations.
- Ensure that all actions are shown in the diagram and are appropriately linked to agents, consistent with agent descriptors.